

Blade Echo

1. Project Overview

Provide a brief overview of your project, explaining the game's concept, mechanics, and functionality.

- Blade Echo is a top-down slashing-and-parry roguelike game built with Pygame. The player navigates procedurally generated maze rooms, collecting and upgrading swords while relying on a core parry mechanic to survive. There are no save files—one death restarts the entire run.
- The dungeon is split into 3 floors, each containing 3 sub-levels. Every sub-level randomly generates 8–10 rooms, and each room is assigned a random event: Monster, Item, Shop, or Special Occurrence. The final sub-level of each floor ends with a randomised boss encounter.

2. Project Review

Study and then present a relevant existing project, outlining the specific improvements you plan to make. Highlight the key areas where your project enhances or builds upon the original.

- The game draws progression inspiration from Soul Knight, a top-down roguelike shooter. However, Blade Echo differentiates itself by making parrying the central mechanic rather than ranged gunplay
 - Parrying ranged enemies deflects bullets back at them for heavy damage.
 - Parrying melee enemies causes a flinch (stun), opening a damage window.
 - The single equipped sword can be upgraded as the player progresses deeper.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

Main mechanic:

- Parry: we're using parry as the main mechanic to weaken the enemy. By deflecting the bullet, the enemy receives a huge amount of damage, in the same way, a flinched enemy takes more damage. You can upgrade your parry in the shop by making the parry window bigger, make the parry radius bigger, or making the weakened multiplier go higher. While these are the basic upgrade, there will also be a major upgrade for the parry mechanic, for example: increase movement speed after parrying, recover health point in a percent, or instant kill the attacker of this attack(of course this doesn't work with bosses).
- Slash: Slashing is the main damage dealing method in the game, as the game progresses, you can make the blade deal more damage and also make the attack size bigger. Like parrying there will be also major upgrade for the sword, for example: dealing flame damage, slowing enemy movement, executing enemy with lower than some hp percent, etc.,
- Enemy: The enemies in the lower floor are easy enemies as these guys are used to familiarize players with some attack, they only have some hp and attack. As the floor progressed the enemy would also come with more diverse attacks, for example: shooting more bullets or the melee enemy with some dash skill. The boss will have their normal attack, but when they reach a certain health point threshold

they will enter a phase where the player can't attack them and can only survive by parrying the hit.

Objectives:

- The goal is to survive from the first floor to the last floor clearing the game.

3.2 Object-Oriented Programming Implementation

List and describe **at least five classes** you will implement in your game. For each class, briefly explain its role, key attributes, and methods. You can also include a UML diagram for better understanding. (Later, this UML will be required as part of the full proposal.)

Class	Role	Key Attributes	Key Methods
Player	The melee protagonist the user controls.	hp, stamina, is_swinging, parry_active, gold, position	attack(), perform_parry(), move(), take_damage(), collect_item()
Sword	Weapon currently equipped by the player.	damage, reach_pixels, swing_arc, knockback, element_type	get_hitbox(), upgrade(), apply_effect()
Enemy (base)	Abstract base class for all enemy types.	hp, speed, attack_damage, state (idle/aggro/flinched), attack_windup_frames	telegraph_attack(), take_damage(), update_ai(), drop_loot()

MeleeEnemy (extends Enemy)	Close-range enemy that charges and strikes.	dash_speed, dash_cooldown	dash_attack(), enter_aggro()
RangedEnemy (extends Enemy)	Projectile-based enemy, keeps distance.	bullet_speed, bullet_count, fire_rate	shoot(), create_bullet_pattern()
Boss (extends Enemy)	Floor-end boss with multiple phases.	phase, phase_threshold, is_invulnerable, attack_patterns	enter_phase(), select_pattern(), spawn_minions()
Room	Single room inside a sub-level.	event_type (monster/item/shop/special), is_locked, walls, enemies_list	generate_layout(), spawn_event(), unlock()
Floor	Manages a full floor of 3 sub-levels.	floor_number, sub_levels, difficulty_modifier	generate_sub_levels(), get_boss_room()
Shop	In-room shop where player buys upgrades.	inventory (list of ShopItem), prices	display_items(), purchase(), refresh_stock()
Item	Collectible or consumable found in rooms.	name, effect_type, value, is_consumable	apply_effect(), describe()

Bullet	Projectile fired by ranged enemies or deflected.	position, velocity, damage, is_deflected	move(), reflect(), check_collision()
CollisionHandler	Central manager for all hitbox checks.	active_hitboxes, active_bullets	detect_clash(), calculate_parry_window(), check_bullet_deflect()
GameManager	Top-level controller for game state.	current_floor, current_room, game_state, run_stats	start_run(), next_room(), game_over(), save_stats()
StatsRecorder	Logs per-run statistics to CSV.	csv_path, current_run_data	log_parry(), log_damage(), log_purchase(), write_row()
HUD	Heads-up display drawn each frame.	health_bar, stamina_bar, gold_display, minimap	draw(), update(), flash_damage()

3.3 Algorithms Involved

Mention the algorithms used in the game. If your game incorporates techniques such as pathfinding, sorting, AI behavior, rule-based logic, or event-driven mechanics, describe them here.

•Hitbox Intersection

- Uses `pygame.Rect.colliderect` and circular collision math to determine if the sword arc overlaps an enemy hurtbox.

•Melee Enemy Attack

– the enemy will try to walk toward the player if there is less than 2 enemies close to the player, each melee enemy has a cooldown of 5 seconds to perform a dash attack.

•Ranged Enemy Attack

– the enemy will try to move further to the player where the attack pattern and cooldown will change depending on the floor that the player resides in.

•Frame-Window Logic

– A rule-based timing system that grants invincibility frames if the parry key is pressed within a 100–200 ms window before an incoming attack lands.

•Randomised DFS Maze Generation

– The maze is generated using a Depth-First Search (DFS) algorithm with randomised neighbour selection. Starting from an initial cell, the algorithm randomly picks an unvisited neighbouring cell, carves a passage to it, and recurses. When it reaches a dead end, it backtracks until it finds a cell with unvisited neighbours. This produces a spanning tree of rooms with exactly one path between any two rooms, ensuring full connectivity. After the maze structure is built, 8–10 rooms per sub-level are selected and each is assigned a randomised event type (Monster, Item, Shop, or Special).

•A* Pathfinding

– Enemy AI uses A* pathfinding on the room grid to navigate around walls toward the player.

4. Statistical Data (Prop Stats)

4.1 Data Features

*Describe **at least five features** you will track in the game, which may include player behavior or game-related metrics (e.g., character movement, keystrokes, pixels moved, player score, time played, enemies defeated,*

accuracy, and completion rate). Each feature should have **at least 100 rows of data**. (For example, in football, a player's performance can be analyzed by collecting and analyzing statistical data.)

	Why is it good to have this data? What can it be used for.	How will you obtain the 100 value of this data?	Which variable (and which class will you collect this from?)	How will you display this feature data?
Survived Time	Tracks how long the player survives; useful for measuring improvement and identifying if later floors take disproportionately long.	Logged automatically per run (each death or clear = 1 row).	survived_time (float) in StatsRecorder; timer starts in GameManager.start_run() and stops on death or game clear.	Bar Chart (Graph 1): Run Number vs. Survived Time. Summary Table: best vs. average.
Parry Streak	Measures mastery of the core parry mechanic.	Logged per run. The counter increments on each successful parry and resets on hit; the max value is saved at run end.	parry_streak (int) in StatsRecorder; updated by CollisionHandler.calculate_parry_window() on each parry event.	Grouped Bar Chart (Graph 3): Floor vs. Successful Parries and Missed Parries side-by-side. Summary Table: best vs. average.
Missed Parries	Measure mastery of parry for player.	Logged per run. Incremented each time the parry key is pressed outside	missed_parries (int) in StatsRecorder; incremented by	Grouped Bar Chart (Graph 3): shown alongside Parry Streak per floor.

		the valid timing window.	CollisionHandler when parry fails.	Summary Table: best vs. average.
Damage Dealt	Shows total offensive output per run; useful for tracking whether the player is improving at killing enemies efficiently.	Logged per run. Accumulated every time Player.attack() or a deflected bullet hits an enemy.	damage_dealt (float) in StatsRecorder; updated by CollisionHandler.detect_clash() on each hit.	Line Graph (Graph 4): Run Number vs. Damage Dealt to show improvement trend. Summary Table: best vs. average.
Money Spent	Reveals player spending priorities (healing vs. sword vs. parry upgrades); helps balance shop economy.	Logged per run. Incremented on each Shop.purchase() call.	money_spent (int) in StatsRecorder; updated by Shop.purchase() with the item price.	Pie Chart (Graph 2): sections for Healing, Sword Upgrades, Parry Upgrades. Summary Table: best vs. average.
Enemies Killed	Tracks combat effectiveness; combined with Damage Dealt it reveals whether the player is overkilling or being efficient.	Logged per run. Incremented on each enemy death event when Enemy.hp reaches 0.	enemies_killed (int) in StatsRecorder; incremented by Enemy.take_damage() when hp <= 0.	Histogram (Graph 5): distribution of enemies killed across all runs. Summary Table: best vs. average.

4.2 Data Recording Method

Explain how the game will store statistical data. Will it be saved in a database, CSV file, or another format?

- Data will be stored in a **CSV file** for easy access and manipulation.

4.3 Data Analysis Report

Outline how you will analyze the recorded data. What statistical measures will you use? How will the analysis be presented (e.g., graphs, tables, charts)?

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Survived Time	Show how long each run lasted to track player improvement over time	Bar Chart	Run Number	Survived Time (seconds)
Graph 2	Money Spent	Show economy breakdown by spending category across all runs	Pie Chart	Sections: Healing, Sword Upgrades, Parry Upgrades	—
Graph 3	Parry Streak, Missed Parries	Compare successful vs. failed parries per floor to identify difficulty spikes	Grouped Bar Chart	Floor (1 / 2 / 3)	Count
Graph 4	Damage Dealt	Show cumulative damage trend over successive runs to track player improvement	Line Graph	Run Number	Damage Dealt (HP)

Graph 5	Enemies Killed	Show distribution of total enemies killed across all runs	Histogram	Enemies Killed (count)	Frequency (runs)
---------	----------------	---	-----------	------------------------	------------------

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Core Melee Mechanics: Implement player movement, sword swing hitboxes, and the basic "Parry" timing window.
3	10	Map Generation & Navigation: Develop the random maze generator for 8–10 rooms and implement enemy A* pathfinding.
4	11	Enemy & Boss Logic: Create varied melee enemy types and the randomized floor-end boss behaviors.
5	12	Economy & Items: Implement the shop system, gold collection, and collectible sword upgrades.
6	13	Data Logging & UI: Setup the CSV recording for parry streaks and damage; design the HUD (Health/Stamina/Stamina).
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 12 March 2025

Editor: Suwat Teiwasuculmetri

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	You still cannot clarify the oop usage well enough. There's still lots of class needed missing. I have a technique for you. First, you describe your system with easy monkey words. For example, Game generates rooms. Room can have enemy, shop or weapon. Shop sell weapon and Item. Player have weapon. Deep room have Boss Then you highlight all the nouns, then you can cover most of the classes. Also, you can see the relationships this way too
18/03	Amornrit	Very clear and detailed game design! As P'Nattanan mentioned, your current class design does not fully cover all the features you proposed. Melee and ranged enemies might behave differently; you may want to model them using inheritance. In data features, explain more about Damage Dealt and Money Spent (what does a 'match' mean?). Collecting the Time Usage feature can be a tedious process (≥ 100 runs).